

JotFlow System Architecture and API Guide

JotFlow

System Architecture and API Guide — Junior Developer Edition
Prepared 19 July 2026

Security rule: Do not put a real JotForm API key in this document, source code, browser URL, screenshot, Git repository, or frontend application. Keep it only in `backend/.env`. If a key was shared publicly, rotate it in JotForm immediately.

1. What this project does

JotFlow is an internal dashboard for JotForm Enterprise workflows. It reads forms, submissions, workflow tasks, and assigned-form prefills from JotForm; stores a usable copy in PostgreSQL; and lets the correct signed-in user approve, reject, complete, or fill a task.

Simple language: JotForm is the workflow engine. JotFlow is the secure control panel that makes the workflow easy to see and operate.

2. Architecture at a glance

Browser (React dashboard) | session cookie; never receives JotForm API key v Express backend (Node.js) |-- validates user, role, request body and task ownership |-- calls JotForm Enterprise API with APIKEY request header |-- reads/writes PostgreSQL |-- sends Socket.IO updates to the browser v JotForm Enterprise API PostgreSQL forms / submissions / workflow / prefills cached dashboard data, users, workflow tasks, history, sessions

Layer	Technology	Responsibility
Frontend	React, TypeScript, Vite, Tailwind, Socket.IO client	Shows the dashboard and sends requests to the backend.
Backend	Express, Zod, Socket.IO, Pino	Security boundary; integrates with JotForm and the database.
Database	PostgreSQL	Stores synchronized submissions, task lists, sessions, users, notifications and history.
Workflow provider	JotForm Enterprise	Owns forms, submissions, workflow instances, tasks, outcomes and prefill records.
Deployment	Windows IIS + ARR reverse	IIS exposes the site; Node

Layer	Technology proxy + Node backend	Responsibility normally listens on port 3001; PostgreSQL normally uses 5432.
--------------	---	--

3. The important IDs

ID	Meaning	Where it comes from	Example use
Parent form ID	The form a requester initially submits.	GET /user/forms, then content[].id.	List its submissions.
Submission ID	One request submitted to a parent form.	GET /form/{formId}/submissions, then content[].id.	Find its workflow tasks.
Workflow instance ID	One running copy of the workflow for a submission.	GET /submission/{submissionId}?addWorkflowStatus=1, then content.workflowInstanceId.	Inspect the full workflow task list.
Task ID	One action assigned to a person.	GET /workflow/submission/{submissionId}/tasks or instance task list, then task.id.	Build assigned-form link or complete the task.
Assigned task form ID	The child form used by an workflow_assignment_form step.	Workflow task: element.internalFormID.	Read the prefill records.
Prefill URL ID	The per-submission token used in the child-form URL.	GET /form/{assignedFormId}/prefills, nested urls[].id.	Build the form-fill link.

The starting point: The first API call in a full project sync is GET /user/forms. It returns form IDs. From a form ID, JotFlow obtains submissions. From a submission, it obtains tasks. Do not start by guessing a workflow ID.

4. Base URL and authentication

Configured API base URL for this deployment:

`https://eforms.mediaoffice.ae/API`

Configured browser/form host:

`https://eforms.mediaoffice.ae`

Every direct JotForm API request in Postman uses this header:

APIKEY: <PUT_YOUR_JOTFORM_API_KEY_HERE>

For the configured team-scoped user profile, use `teamID` as a query parameter where shown below. The backend adds it automatically. Some endpoints also accept the following header in the current integration:

`jf-team-id: 260541093809054`

Never add APIKEY to a browser task URL. API keys authorize server-to-server API calls. A task recipient URL has a different authorization model: a prefill ID for assigned forms, or an issued access/share token for approval/task links.

5. Full discovery flow: from zero to a task URL

Step 1 — List all forms (first call)

GET `https://eforms.mediaoffice.ae/API/user/forms?limit=1000&teamID=260541093809054`
APIKEY: <YOUR_API_KEY>

Find the parent workflow form. Save its `content[].id` as `PARENT_FORM_ID`. In an enterprise-scoped JotForm API profile, the equivalent path may be `/API/enterprise/forms`; this project automatically chooses the correct scope from its profile configuration.

Step 2 — List submissions for the parent form

GET `https://eforms.mediaoffice.ae/API/form/{PARENT_FORM_ID}/submissions?limit=1000&offset=0&orderby=created_at&direction=DESC&addWorkflowStatus=1`
APIKEY: <YOUR_API_KEY>

Each `content[].id` is a `SUBMISSION_ID`. The descending order requests newest first. Do not assume `content[0]` is the right request without confirming the requester, date, and form answers.

Step 3 — Get task list for one submission

GET `https://eforms.mediaoffice.ae/API/workflow/submission/{SUBMISSION_ID}/tasks`
APIKEY: <YOUR_API_KEY>

This is the preferred task lookup. Find a task where `status` is `ACTIVE` and its assignee email matches the person who should act. The task ID is `id`.

```
{
  "id": "TASK_ID_TO_USE",
  "status": "ACTIVE",
```

```

    "element": { "type": "workflow_assign_form", "internalFormID":
"CHILD_FORM_ID" },
    "properties": { "assigneeEmail": "person@example.com" }
}

```

Optional Step 3b — Get workflow instance detail

Use this when you need the full instance, resource shares, or the submission endpoint did not return tasks.

```

GET https://eforms.mediaoffice.ae/API/submission/{SUBMISSION_ID}?
addWorkflowStatus=1
APIKEY: <YOUR_API_KEY>

```

Then read `content.workflowInstanceID`, and call:

```

GET
https://eforms.mediaoffice.ae/API/workflow/instance/{WORKFLOW_INSTANCE_ID}

```

Step 4 — Get prefill records for an assigned-form task

```

GET https://eforms.mediaoffice.ae/API/form/{CHILD_FORM_ID}/prefills?
limit=1000&offset=0
APIKEY: <YOUR_API_KEY>
jf-team-id: 260541093809054

```

The response contains prefill templates in `content[]`, with per-submission records nested in `content[].urls[]`. Select the record where:

```
urls[].settings.id === SUBMISSION_ID
```

Use its nested `urls[].id`, not the outer `content[].id` and not `urls[].prefill_id`.

Step 5 — Build the assigned-form browser URL

```

https://eforms.mediaoffice.ae/{CHILD_FORM_ID}/prefill/{PREFILL_URL_ID}?
workflowAssignFormTask=1&taskID={TASK_ID}

```

Example structure only:

```

https://eforms.mediaoffice.ae/260825008947058/prefill/
6a5b77c3663633bf7e959cde9126?workflowAssignFormTask=1&taskID=TASK_ID

```

This is a **browser URL**, not a Postman API endpoint. It is valid only for an active `workflow_assign_form` task and the matching parent submission.

6. Task types and what URL can be created

Task type	What to open	Can it be built from IDs alone?
<code>workflow_assign_form</code>	Prefilled child form URL.	Yes, when the Prefills API provides a matching nested <code>urls[].id</code> .
<code>workflow_approval</code>	JotForm approval page.	No. It normally requires the access/share token JotForm

Task type	What to open	Can it be built from IDs alone?
workflow_assign_task	JotForm task link.	issued. API key must not be put in the URL. Usually no. Use JotForm's returned access/share link or the app's email-link resolver.

7. Completing, approving, or rejecting a task

The browser calls the app, not JotForm directly:

```
POST /api/workflow-action
Content-Type: application/json

{
  "submissionId": "SUBMISSION_ID",
  "taskId": "TASK_ID",
  "action": "approve",
  "comment": "Optional comment",
  "signature": "Optional signature value"
}
```

The backend verifies the user session, finds the active task assigned to that user, determines the JotForm outcome ID, then sends this protected server-to-server request:

```
POST https://eforms.mediaoffice.ae/API/workflow/task/{TASK_ID}/complete
APIKEY: <SERVER_SIDE_API_KEY>
Content-Type: application/json

{ "outcomeID": 1, "comment": "Optional comment", "signature": "Optional signature value" }
```

Do not let the frontend call this completion API with the API key. It would expose a powerful secret to every browser user.

8. Application APIs used by the React dashboard

Method and route	Who can use it	Purpose
GET /api/health	Public	Basic liveness check.
GET /api/health/ready	Public	Checks PostgreSQL as well as the server.
GET /api/jotform?path=user/forms	Signed-in user	Safe backend proxy for a small allowlist of JotForm metadata routes.
GET /api/submissions	Signed-in user	Reads stored, permission-filtered dashboard submissions from PostgreSQL.
GET /api/workflow-tasks?	Signed-in user	Gets and flattens current workflow tasks for one

Method and route	Who can use it	Purpose
submissionId=...		submission.
GET /api/email-url?formId=...&submissionId=...&taskId=...	Signed-in user	Resolves a safe task/action link, including stored email/share links where available.
POST /api/workflow-action	Signed-in assigned user	Approve, reject, or complete the active task.
POST /api/admin/sync-all	Admin	Re-sync forms and submissions from JotForm.
GET /api/admin/sync-all-stream	Admin	Same sync with server-sent progress events.

9. Background synchronization and live updates

1. JotForm can send a webhook when a submission is created or changed.
2. The backend fetches the authoritative submission and workflow task state.
3. The poller also periodically lists forms and submissions, so the app can recover if a webhook is missed.
4. The backend saves normalized data to PostgreSQL.
5. The backend emits `submissions:updated` through Socket.IO.
6. The React dashboard reloads its database-backed view.

For assigned-form tasks, JotForm may create the prefill record shortly after the task appears. The backend checks the Prefills API and marks the task `pending`, `ready`, or `not_required`. It does not open a normal bare child-form URL while a configured prefill is still pending.

10. Postman checklist

1. Create a request with method **GET**.
2. Use the exact API URL from the relevant step.
3. Add `APIKEY` under Headers. Do not use Authorization Bearer unless JotForm explicitly requires it.
4. For team-scoped calls, add `teamID=260541093809054` to the query string. For the prefill endpoint, also use `jf-team-id` if your account requires it.
5. Do not test from a browser-only API client if it reports CORS/network errors; use Postman desktop, Hoppscotch Proxy/Agent, or curl. This is a browser restriction, not necessarily a bad URL.
6. Never paste a live key into a chat, screenshot, ticket, or shared collection.

11. Troubleshooting map

Symptom	Likely cause	Check first
401 or 403 from JotForm	Wrong/expired key, wrong team scope, or insufficient permission.	APIKEY header; profile scope; teamID.

Symptom	Likely cause	Check first
Task list is empty	Wrong submission ID, workflow is not created yet, or endpoint shape differs.	/workflow/ submission/ {submissionId}/tasks, then instance lookup fallback.
No matching prefill	Prefill is still being created or the wrong parent submission was used.	Match urls[].settings.id exactly to the parent submission ID; retry shortly.
Assigned-form URL opens wrong data	Used the outer prefill template ID instead of nested URL ID.	Use urls[].id only.
Approval link has access denied	Approval/task link needs JotForm-issued access token or share link.	Use /api/email-url or stored JotForm access link; do not append APIKEY.
Hoppscotch says network/CORS error	Browser cannot call the API cross-origin.	Use Postman desktop or Hoppscotch Proxy/Agent.

12. Key source files

File

backend/lib/jotform.js

backend/lib/profiles.js

backend/lib/prefill.js

backend/lib/workflow-task.js

backend/routes/submissions-
actions.js

backend/routes/admin-sync.js

backend/routes/forms-
workflow.js

backend/app.js

frontend/src/pages/
ModernDashboard.tsx

Why a junior should know it

One shared JotForm HTTP client; adds APIKEY header, profile scope, team ID and timeout.

Defines which JotForm host, API base URL, team and secret each profile uses.

Reads Prefills API, matches nested URL records to a submission, and builds assigned-form URLs.

Converts JotForm's nested task JSON into fields used by the frontend.

Reads tasks and safely completes approve/reject actions.

Full data synchronization from forms to submissions to workflow tasks.

Resolves browser-safe action and task links.

Express middleware, security controls and all route mounting.

Main personal action queue.

13. Final rule of thumb

Use IDs to discover data. Use the backend to protect secrets. Use issued task links to open tasks.

Form ID → Submission ID → Task ID → (for assigned forms) Prefill URL ID → Browser URL.